00 I file di un progetto Lazarus

Versione 1.0 — 28/10/2026 — documento per il docente

Classe 1, prima lezione con Lazarus 4 su Windows. Progetto di esempio: «ciao», con un solo form (unit1).

1 Un progetto = una cartella

1. Un progetto Lazarus è **una cartella**: dentro ci sono tutti i file che servono al programma.
2. Ogni file ha un'**estensione**: le lettere dopo il punto nel nome (in `unit1.pas` l'estensione è `.pas`). L'estensione dice **che cosa c'è dentro** il file e quale programma lo apre.
3. Windows di solito nasconde le estensioni. Per vederle, in Esplora file: menu **Visualizza** → **Mostra** → spunta **Estensioni nomi file**.
4. La regola da ricordare: alcuni file li scriviamo noi (sono il nostro lavoro e si **tengono**); altri li costruisce Lazarus da solo ogni volta (se si cancellano, **si rifanno**).

2 .lpi: la carta d'identità del progetto

1. **Che cos'è:** Lazarus Project Information (informazioni sul progetto Lazarus). Nel nostro esempio: `ciao.lpi`.
2. **A che cosa serve:** è la **carta d'identità** del progetto. Dice a Lazarus quali file fanno parte del progetto e come va costruito. È il file da aprire con **doppio clic** per riprendere il lavoro: si apre Lazarus con tutto il progetto.
3. **Che cosa c'è dentro:** l'elenco dei file del progetto e tutte le **opzioni del progetto e del compilatore** (il compilatore è il programma che traduce il nostro codice nel linguaggio del computer): titolo dell'applicazione, icona, nome e cartella dell'exe, se mettere le informazioni per trovare gli errori. Sono le impostazioni che si cambiano da **Progetto → Opzioni progetto**. È un file di testo (formato XML, testo con etichette tra parentesi angolari).
4. **Quando viene creato:** al primo salvataggio del progetto; si aggiorna quando si cambiano le opzioni.
5. **Si può cancellare? No.**
6. **Se manca:** non c'è più il progetto da aprire con doppio clic e si perdono tutte le opzioni. I file del codice restano, ma il progetto va ricostruito a mano.

3 .lpr: il programma principale

1. **Che cos'è:** Lazarus PRogram (programma Lazarus). Nel nostro esempio: `ciao.lpr`.
2. **A che cosa serve:** è il **punto di partenza**, come la **porta d'ingresso** di una casa: quando il programma parte, entra da qui. Accende l'applicazione, crea la finestra (il form) e la mostra.
3. **Che cosa c'è dentro:** poche righe di codice Pascal scritte da Lazarus (`program ciao;`, l'elenco delle unit usate, `Application.Initialize`, `Application.CreateForm`, `Application.Run`). Di solito **non si tocca** a mano.
4. **Quando viene creato:** al primo salvataggio del progetto, con il nome scelto per il progetto.
5. **Si può cancellare? No.**
6. **Se manca:** il programma non si può più compilare (costruire): manca la porta d'ingresso.

4 .pas (e .pp): il codice che scriviamo noi

1. **Che cos'è:** file sorgente **Pascal**, cioè il testo del programma. Nel nostro esempio: `unit1.pas`. L'estensione `.pp` (Pascal) è la stessa cosa: un altro nome per i file Pascal, usato da Free Pascal.
2. **A che cosa serve:** è il **quaderno** dove scriviamo le istruzioni: che cosa succede quando si clicca un bottone, che cosa si scrive in una casella.
3. **Che cosa c'è dentro:** la **unit** (un pezzo di programma) collegata al form: le dichiarazioni dei componenti e le procedure degli eventi, per esempio `Button1Click`.
4. **Quando viene creato:** quando si crea il form e lo si salva (Lazarus chiede il nome: `unit1`).
5. **Si può cancellare? No:** è il nostro lavoro.
6. **Se manca:** si perde tutto il codice scritto; il progetto non si compila.

5 .lfm: il disegno del form

1. **Che cos'è:** Lazarus ForM (form di Lazarus). Nel nostro esempio: `unit1.lfm`. Ha sempre lo stesso nome della sua `.pas`: vanno **in coppia**.
2. **A che cosa serve:** è la **foto della finestra** come l'abbiamo disegnata: dove sta il bottone, quanto è grande, che scritta ha.
3. **Che cosa c'è dentro:** testo con l'elenco dei componenti e le loro **proprietà** (quelle dell'Ispettore oggetti): `Left`, `Top`, `Width`, `Height`, `Caption`, e il collegamento agli eventi (`OnClick = Button1Click`).
4. **Quando viene creato:** insieme alla `.pas`, quando si salva il form; si aggiorna ogni volta che si sposta o si cambia un componente.
5. **Si può cancellare? No.**
6. **Se manca:** la finestra perde tutti i componenti disegnati; il codice della `.pas` non trova più i suoi bottoni e il progetto non funziona.

6 .lps: la sessione (il segnalibro)

1. **Che cos'è:** Lazarus Project Session (sessione del progetto Lazarus). Nel nostro esempio: `ciao.lps`.
2. **Che cos'è la sessione:** è **dove eri rimasto** l'ultima volta che hai lavorato. Come il **segnalibro** in un libro, o il salvataggio di un videogioco che ti rimette nel punto in cui avevi lasciato.
3. **Che cosa c'è dentro:** quali file erano aperti nell'editor (le schede), la **posizione del cursore** in ogni file, i **punti di interruzione** (i punti rossi dove il programma si ferma per controllare), la disposizione dell'editor. **Non contiene il programma.**
4. **Quando viene creato:** quando si salva o si chiude il progetto; si riscrive ogni volta.
5. **Si può cancellare? Sì.**
6. **Se manca:** il programma è intatto; alla prossima apertura Lazarus non ricorda quali schede erano aperte né dove era il cursore. Il file **si rifà** da solo.

7 .res: le risorse

1. **Che cos'è:** RESource (risorsa). Nel nostro esempio: `ciao.res`.
2. **A che cosa serve:** è la **valigia degli accessori** del programma: cose che non sono codice ma vanno dentro l'exe.
3. **Che cosa c'è dentro:** l'**icona**, le informazioni di versione e altre impostazioni per Windows. È un file binario (non si legge con Blocco note).
4. **Quando viene creato:** da Lazarus, a partire dalle opzioni del progetto.
5. **Si può cancellare? Sì.**
6. **Se manca:** Lazarus lo **rifà** alla compilazione successiva.

8 .ico: l'icona

1. **Che cos'è:** ICOn (icona). Nel nostro esempio: `ciao.ico`.

2. **A che cosa serve:** è il **disegnino** del programma, quello che si vede sul file exe e sulla barra delle applicazioni.
3. **Che cosa c'è dentro:** un'immagine piccola in formato icona di Windows.
4. **Quando viene creato:** quando si crea il progetto (icona standard di Lazarus); si cambia da **Progetto → Opzioni progetto → Applicazione**.
5. **Si può cancellare? Sì**, se è quella standard.
6. **Se manca:** il programma funziona lo stesso; l'icona si può scegliere di nuovo dalle opzioni del progetto.

9 .exe: il programma finito

1. **Che cos'è:** EXEcutable (eseguibile). Nel nostro esempio: `ciao.exe`.
2. **A che cosa serve:** è il **piatto pronto**: i file `.pas` e `.lfm` sono la ricetta, l'exe è il piatto cucinato. Si avvia con doppio clic, anche senza Lazarus.
3. **Che cosa c'è dentro:** il programma tradotto nel linguaggio del computer (codice macchina, illeggibile per noi), con form, icona e risorse già dentro.
4. **Quando viene creato:** ogni volta che si preme **Esegui** (il triangolo verde) o **Esegui → Compila**.
5. **Si può cancellare? Sì.**
6. **Se manca:** si rifà premendo di nuovo Esegui.

10 La cartella lib\x86_64-win64\: il laboratorio del compilatore

1. **Che cos'è:** una cartella creata dal compilatore. Il nome dice per quale computer costruisce: `x86_64` = processore a 64 bit, `win64` = Windows a 64 bit.
2. **A che cosa serve:** è il **banco di lavoro** dove il compilatore lascia i pezzi intermedi prima di montare l'exe. Come i semilavorati in un laboratorio: servono a fare prima la volta dopo.
3. **I file dentro:**
    1. **.o**: file **oggetto** (object). Il codice di una unit già tradotto in linguaggio macchina, ma non ancora unito agli altri pezzi. Esempio: `unit1.o`.
    2. **.ppu**: la **unit compilata**. Contiene la descrizione della unit già pronta per il compilatore, così la volta dopo non deve rileggere tutto da capo. Esempio: `unit1.ppu`.
    3. **.compiled**: un promemoria di Lazarus con le **impostazioni dell'ultima compilazione**; serve a capire se bisogna ricompilare tutto. Esempio: `ciao.compiled`.
    4. **.or**: le **risorse compilate** (object resource): le risorse del programma trasformate in un pezzo pronto da unire nell'exe.
4. **Quando viene creata:** alla prima compilazione (primo clic su Esegui).
5. **Si può cancellare? Sì**, tutta la cartella `lib`.
6. **Se manca:** alla compilazione successiva il compilatore la **rifà**; la prima volta ci mette solo un po' di più.

11 La cartella backup\: le copie di sicurezza

1. **Che cos'è:** una sottocartella che si chiama `backup`, creata da Lazarus.
2. **A che cosa serve:** è il **paracadute**: ogni volta che si salva, Lazarus mette qui la versione **precedente** del file. Se si rovina qualcosa, si può recuperare la copia di prima.
3. **Che cosa c'è dentro:** con le impostazioni normali di Lazarus 4, i file del progetto vengono copiati con **lo stesso nome** (per esempio `backup\unit1.pas`, `backup\unit1.lfm`). I file che non fanno parte del progetto vengono copiati aggiungendo l'estensione **.bak** (per esempio `appunti.txt.bak`). Le impostazioni sono in **Strumenti → Opzioni → Ambiente → Backup**.
4. **Quando viene creata:** al secondo salvataggio di un file (al primo non c'è ancora una versione precedente da copiare).

5. **Si può cancellare? Sì.**
6. **Se manca:** il progetto funziona uguale; si perdono solo le copie delle versioni vecchie.

12 Tabella riassuntiva

Estensione	Che cos'è	Tenere o si rifà
.lpi	Carta d'identità del progetto e opzioni (si apre con doppio clic)	**TENERE**
.lpr	Programma principale (la porta d'ingresso)	**TENERE**
.pas / .pp	Codice Pascal che scriviamo noi (la unit)	**TENERE**
.lfm	Disegno del form (componenti e proprietà)	**TENERE**
.lps	Sessione: dove eri rimasto (segnalibro)	si rifà
.res	Risorse: icona e informazioni di versione	si rifà
.ico	Icona del programma	si rifà
.exe	Programma finito da avviare	si rifà
lib\ (.o .ppu .compiled .or)	Pezzi intermedi del compilatore	si rifà
backup\	Copie di sicurezza delle versioni precedenti	si rifà

La regola: si tengono (e si consegnano o si mettono su GitHub) **solo** `.lpi`, `.lpr`, `.pas`, `.lfm`. Tutto il resto Lazarus lo **rifà** da solo.

13 Da scrivere alla lavagna

```
ciao\                  <- UNA cartella = UN progetto
 |-- ciao.lpi         TENERE   (doppio clic: apre il progetto)
 |-- ciao.lpr         TENERE   (programma principale)
 |-- unit1.pas        TENERE   (il codice che scrivo io)
 |-- unit1.lfm        TENERE   (il disegno del form)
 |-- ciao.lps         si rifa' (dove ero rimasto)
 |-- ciao.res         si rifa' (risorse: icona, versione)
 |-- ciao.ico         si rifa' (icona)
 |-- ciao.exe         si rifa' (il programma finito)
 |-- lib\x86_64-win64\
 |                    si rifa' (.o .ppu .compiled .or)
 |-- backup\          si rifa' (copie di sicurezza)

DA CONSEGNARE:  .lpi  .lpr  .pas  .lfm
```